

Manual de Instalación ScanGo

Aplicación de autocompra para tiendas físicas

Nome Alumno/a:Alba Rodríguez Fernández

Curso: 2º DAM

Módulo: *Proyecto Final Ciclo*

Índice

1.Introducción	1
2. Arquitectura do sistema	2
3. Requisitos do sistema	3
4. Obtención do código fonte	4
5. Instalación do backend (FastAPI)	5
6.Poboación inial da base de datos	8
7.Instalación do Fronted (Flutter)	10
8. Orde de inicio do sistema	12
9. Verificación completa do sistema	13
10. Parada do sistema	14
11. Resolución de problemas frecuentes	15
12.Despregamento en produción	17
Conclusión e seguintes pasos	18

1.Introducción

ScanGo é unha aplicación web de autocompra para tendas físicas. Permite aos clientes escanear produtos cos seus códigos QR, engadilos a un carrito de compra virtual e completar a transacción de forma rápida e autónoma, sen necesidade de pasar por un caixero.

Para quen está pensado este manual

Este manual vai dirixido a técnicos e persoas con coñecementos básicos de programación que precisen instalar e executar ScanGo en contorno local de desenvolvemento ou para probas. Non é necesario ser un experto, pero si ter familiaridade con conceptos como terminais, liña de comandos e instalación de software.

Que inclúe este documento

Explicaremos paso a paso como:

- Instalar todas as ferramentas necesarias
- Descargar o código fonte desde GitHub
- Configurar o servidor backend (FastAPI + Python)
- Inicializar a base de datos
- Iniciar a aplicación frontend (Flutter Web)
- Verificar que todo funciona correctamente
- Resolver problemas comúns

2. Arquitectura do sistema

ScanGo está composto de dúas aplicacións que deben traballar en paralelo:

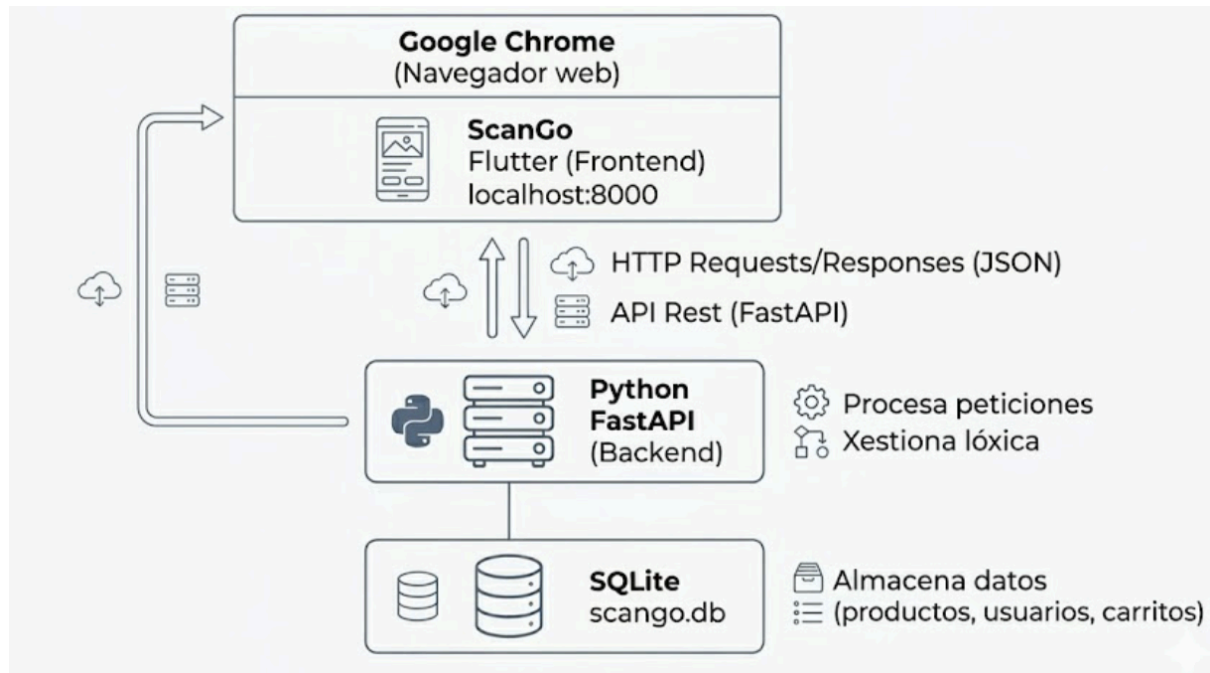
Backend (Servidor)

Executase en Python usando FastAPI. Xestiona a lóxica de negocio, os datos (base de datos), e ofrece endpoints (rutas) que o frontend consulta para obter información de produtos, xestionar carritos, usuarios, etc.

Frontend (Interfaz visual)

É a aplicación que ve o usuario, desenvolvida en Flutter Web. Corre nun navegador (Chrome) e comunícase co backend para solicitar datos e enviar accións.

Flujo de comunicación



3. Requisitos do sistema

3.1 Hardware mínimo

Especificación	Requisitos mínimos
Procesador	Dual-core a 1.5 GHz ou superior
Memoria RAM	4 GB
Espazo no disco	500 MB
Conexión	Rede local (non require internet)

3.2 Software necesario

Asegúrate de instalar primeiro Python, despois Git, e finalmente Flutter SDK.

Ferramenta	Versión mínima	Descarga oficial
Python	3.10 +	https://www.python.org/downloads/
Git	2.30 +	https://git-scm.com/downloads
Flutter SDK	3.0+	https://flutter.dev/docs/get-started/install
Google Chrome	90+	https://www.google.com/chrome/
Node.js	16+	https://nodejs.org/

3.3 Como verificar a instalación

Abre un terminal (ou PowerShell en Windows) e executa estes comandos para comprobar que todo está instalado:

Python

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● Alba@SanMamede-Apple-01 SCANGO % python3 --version
Python 3.10.12
○ Alba@SanMamede-Apple-01 SCANGO %
```

(A versión exacta pode variar, pero debe ser 3.10 ou superior)

Git

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● Alba@SanMamede-Apple-01 SCANGO % git --version
git version 2.34.1
○ Alba@SanMamede-Apple-01 SCANGO %
```

Flutter

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● Alba@SanMamede-Apple-01 SCANGO % flutter --version
Flutter 3.13.0 • channel stable • framework • revision d211f04860
Engine • revision 2f6c601047
Dart 3.1.0
```

Google Chrome

Simplemente abre Chrome e verifica que está instalado

4. Obtención do código fonte

Opción A: Clonar desde GitHub (recomendado)

Abre un terminal e executa:

```
○ Alba@SanMamede-Apple-01 ScanGo % git clone https://github.com/albarf1/ScanGo.git
```

Isto descargará todo o código do proxecto nunha carpeta chamada ScanGo.

Opción B: Descarga manual

1. Vai a <https://github.com/albarf1/ScanGo>
2. Pulsa o botón verde Code > Download ZIP
3. Descomprime o ficheiro en calquera carpeta
4. Abre un terminal e navega até esa carpeta

Estrutura de carpetas esperada

Despois de clonar, o teu directorio debe quedar tal que así:

```
ScanGo/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── database.py
│   │   ├── models.py
│   │   └── routers/
│   │       ├── productos.py
│   │       ├── carrito.py
│   │       └── usuarios.py
│   ├── __init__.py
│   ├── datosProba.py
│   ├── requirements.txt
│   └── scango.db (crearse máis tarde)
├── frontend/
│   ├── lib/
│   ├── pubspec.yaml
│   └── (máis arquivos Flutter)
└── README.md
```

5. Instalación do backend (FastAPI)

5.1 Acceder ao cartafol backend

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● Alba@SanMamede-Apple-01 SCANGO % cd ScanGo/backend
```

Verifica que estás no lugar correcto executando:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Alba@SanMamede-Apple-01 SCANGO % cd ScanGo/backend
Alba@SanMamede-Apple-01 backend % ls
README.md      __init__.py    app            datosProba.py  requirements.txt  scango.db
```

Deberías ver: app/, datosProba.py, requirements.txt

5.2 Crear e activar o contorno virtual de Python

Un contorno virtual é un aillamento de Python no teu ordenador que evita conflitos de dependencias entre proxectos.

En Mac/Linux

```
[Alba@SanMamede-Apple-01 ~ % cd ScanGo
Alba@SanMamede-Apple-01 ScanGo % python3 -m venv venv
[source venv/bin/activate
(venv) Alba@SanMamede-Apple-01 ScanGo %
```

Verás que aparece (venv) á esquerda do termin

En Windows (PowerShell)

```
python -m venv venv
```

```
.\venv\Scripts\Activate.ps1
```

É importante o contorno virtual guardar as librerías do proxecto en carpeta separada. Se algo falla, simplemente eliminas a carpeta venv e voces a crear.

5.3 Instalar dependencias

Con o contorno virtual activado, executa:

```
[Alba@SanMamede-Apple-01 ~ % cd ScanGo
Alba@SanMamede-Apple-01 ScanGo % pip install -r requirements.txt
```

Saída esperada:

```
Collecting fastapi==0.104.1
  Downloading fastapi-0.104.1-py3-none-any.whl (92 kB)
Collecting sqlalchemy==2.0.23
  Downloading sqlalchemy-2.0.23-cp312-cp312-win_amd64.whl (2.1 MB)
...
Successfully installed fastapi-0.104.1 sqlalchemy-2.0.23 uvicorn-0.24.0
```

Este comando instala as tres librerías principais:

- **FastAPI:** Framework web para crear APIs REST
- **SQLAlchemy:** Ferramenta para traballar con bases de datos
- **Uvicorn:** Servidor que executa a aplicación FastAPI

5.4 Iniciar o servidor FastAPI

Aínda con o contorno virtual activado, executa:

```
[Alba@SanMamede-Apple-01 ScanGo % python3 -m uvicorn app.main:app --reload --port 8000]
INFO: Will watch for changes in these directories: ['/Users/Alba/ScanGo']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [44686] using StatReload
Process SpawnProcess-1:
Traceback (most recent call last):
  File "/Applications/Xcode.app/Contents/Developer/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/multiprocessing/process.py", line 315, in _bootstrap
    self.run()
  File "/Applications/Xcode.app/Contents/Developer/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/multiprocessing/process.py", line 108, in run
    self._target(*self._args, **self._kwargs)
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/_subprocess.py", line 80, in subprocess_started
    target(sockets=sockets)
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/server.py", line 67, in run
    return asyncio_run(self.serve(sockets=sockets), loop_factory=self.config.get_loop_factory())
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/_compat.py", line 60, in asyncio_run
    return loop.run_until_complete(main)
  File "/Applications/Xcode.app/Contents/Developer/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/asyncio/base_events.py", line 642, in run_until_complete
    return future.result()
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/server.py", line 71, in serve
    await self._serve(sockets)
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/server.py", line 78, in _serve
    config.load()
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/config.py", line 439, in load
    self.loaded_app = import_from_string(self.app)
  File "/Users/Alba/Library/Python/3.9/lib/python/site-packages/uvicorn/importer.py", line 22, in import_from_string
    raise exc from None
```

O servidor agora está operativo e escoltando no porto 8000.

Non peches este terminal. O servidor esta executándose continuamente. Abre un novo terminal para os próximos pasos.

5.5 Verificar que o servidor funciona

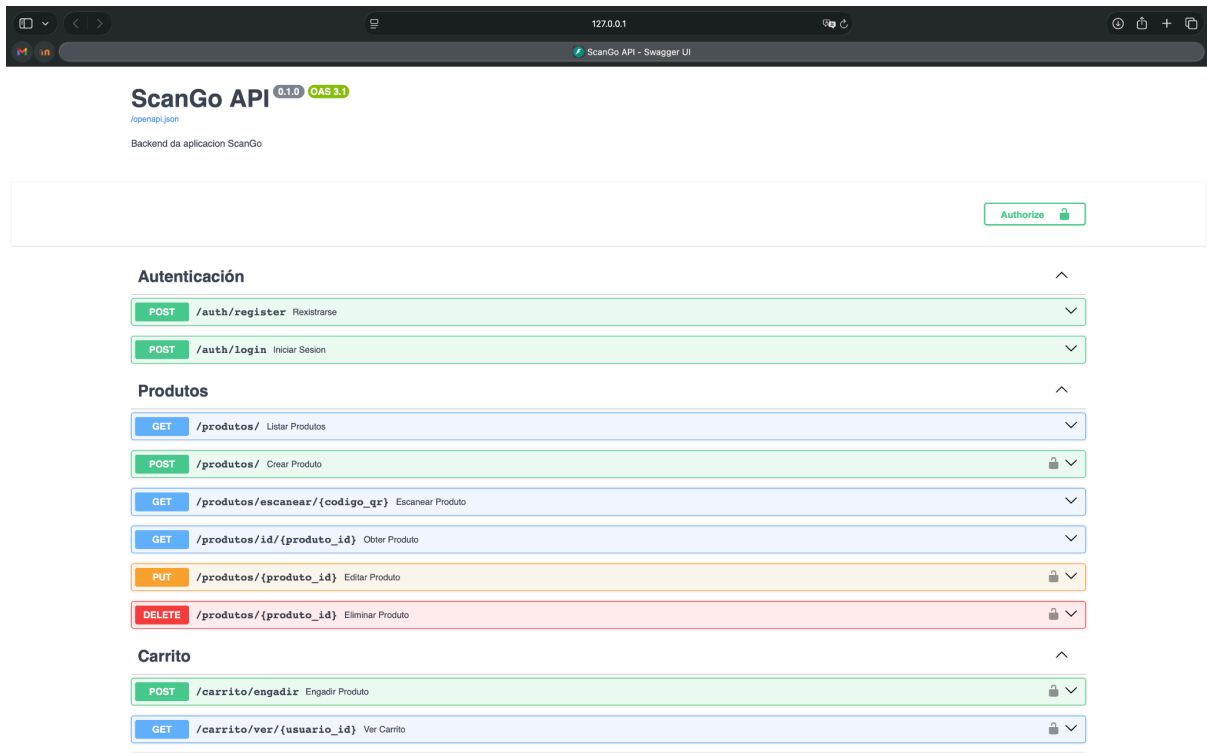
Abre o teu navegador web e vai a: <http://127.0.0.1:8000/docs>



Deberías ver a documentación interactiva de Swagger UI coa lista de todos os endpoints:

- GET /productos/ (listar productos)
- GET /productos/escanear/{codigo_qr} (buscar por QR)
- POST /carrito/engadir (engadir ao carrito)
- GET /carrito/ver/{usuario_id} (ver carrito)
- E moito máis...

Se ves esta páxina, a instalación foi correcta



5.6 Sobre o ficheiro de base de datos

Ao arrancar o servidor, créase automaticamente un ficheiro chamado **scango.db** (se non existe). Este é o ficheiro que garda todos os datos

```
backend/  
├─ scango.db ← Base de datos SQLite (créase automaticamente)  
├─ app/  
├─ datosProba.py  
└─ requirements.txt
```

Se en algún momento queres "limpiar" os datos e comezar de novo, simplemente elimina este ficheiro. A próxima vez que arranques o servidor, créase novamente en branco.

6.Poboación inial da base de datos

Para que a aplicación teña datos con que traballar, executamos un script que crea produtos de exemplo e usuarios de proba.

6.1 Executar o script de datos

Abre un novo terminal (deixando o servidor FastAPI a executar no outro). Navega á carpeta backend (sen entrar no contorno virtual de novo, ou entrando noutro terminal):

```
● Alba@SanMamede-Apple-01 PRUEB % cd ScanGo
● Alba@SanMamede-Apple-01 ScanGo % cd backend
○ Alba@SanMamede-Apple-01 backend % python3 -m venv venv
  source venv/bin/activate # (ou o correspondente a Windows)
  python3 datosProba.py
```

Saída esperada:

```
Creando usuarios...
Usuarios creados correctamente
Creando productos...
8 productos creados correctamente
Creando carritos de proba...
Carritos creados correctamente

Datos de proba cargados
- 8 productos disponibles
- 2 usuarios creados
- 1 carrito con 2 productos
```

6.2 Usuarios e productos creados

Despois de executar o script, a base de datos ten:

Usuarios de proba

Email	Nome	Contraseña	Rol
alba@scango.com	Alba Rodríguez	password123	Usuario
jacobo@scango.com	Jacobo Martinez	password123	Usuario

Productos de exemplo

Código QR	Nome	Prezo	Stock
PROD001	Leite enteiro 1L	€1,29	50
PROD002	Pan integral 500g	€1,99	30
PROD003	Queso semicurado 250g	€3,45	20
PROD004	Aceite de oliva 1L	€6,99	15
PROD005	Auga mineral 6x500ml	€1,50	100
PROD006	Café molido 250g	€2,79	40
PROD007	Iogur natural 125g	€0,89	60
PROD008	Atún en aceite 190g	€1,45	35

Para probas iniciais, usa estes usuarios. Poderás crear máis desde a aplicación frontend.

7.Instalación do Fronted (Flutter)

O frontend é a interface visual que ven os usuarios. Está desenvolvida en Flutter e corre dentro de Chrome como aplicación web.

7.1 Abrir un novo terminal

Deixa o servidor FastAPI executándose no terminal que abriste antes. Non o peches. Abre un terminal novo en mesmo ordenador (podes ter dous terminais abertos á vez).

7.2 Acceder ao cartafol frontend

```

● Alba@SanMamede-Apple-01 PRUEB % cd ScanGo
● Alba@SanMamede-Apple-01 ScanGo % cd frontend
○ Alba@SanMamede-Apple-01 frontend % █

```

Verifica que estás no lugar correcto:

```

● Alba@SanMamede-Apple-01 frontend % ls
README.md      build          lib            pubspec.lock  pubspec.yaml  test          web
○ Alba@SanMamede-Apple-01 frontend % █

```

Deberías ver arquivos como `pubspec.yaml`, `lib/`, etc.

7.3 Instalar dependencias de Flutter

```
Alba@SanMamede-Apple-01 frontend % flutter pub get
```

Saída esperada:

```
Running "flutter pub get" in flutter...
Resolving dependencies...
  provider 6.0.5
  http 1.1.0
  ...
Got dependencies in 8.5s.
```

Este comando descarga todas as librerías que Flutter precisa (widgets, HTTP client, etc.).

7.4 Iniciar a aplicación

```
Alba@SanMamede-Apple-01 frontend % flutter run -d chrome --web-port 3000
```

A primeira execución tardará uns minutos mentres Flutter compila o código TypeScript/JavaScript:

Saída durante compilación:

```
Compiling flutter web app to JavaScript...
Building application for the web...
[Compiling 123 Dart files] [██████████] 2m 15s
[Compiling Kotlin files] [██████████] 45s
```

Saída final esperada:

```
webdev server listening on http://localhost:3000, serving web/
INFO: Hot restart performed in 3.2s.
```

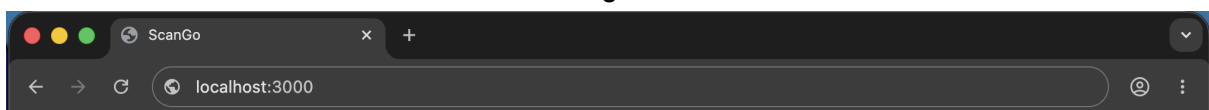
7.5 Processo de compilación

- Primeira vez: Pode tardar 1-3 minutos (Flutter prepara todo o código)
- Seguintes execucións: ~30 segundos (usa caché)

Non peches o terminal. Esta ventá indicará cambios no código en tempo real.

7.6 Abrir a aplicación

Cando vexas a mensaxe anterior, abre o navegador e vai a:

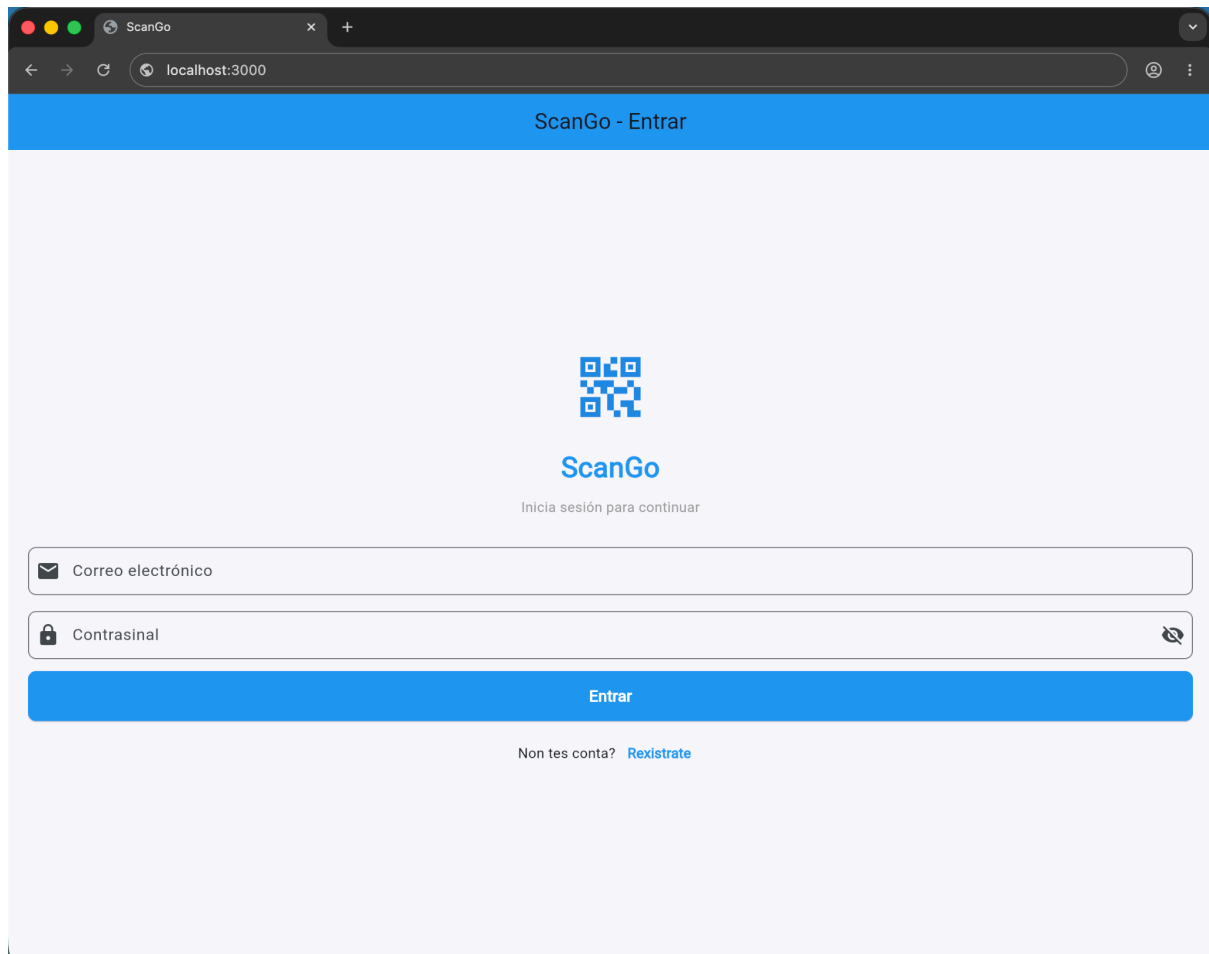


Deberías ver a pantalla de login de ScanGo con:

- Campo para email

- Campo para contraseña
- Botón "Iniciar sesión"

Se ves esta pantalla, a instalación é correcta.



8. Orde de inicio do sistema

Cada vez que queiras usar ScanGo, debes arrancar os dous servizos na orde correcta:

Paso 1: Arrancar o Backend (FastAPI)

```
Alba@SanMamede-Apple-01 ScanGo % cd ScanGo/backend
source venv/bin/activate # (Mac/Linux)
# ou .\venv\Scripts\Activate.ps1 (Windows PowerShell)
# ou venv\Scripts\activate.bat (Windows CMD)

python3 -m uvicorn app.main:app --reload --port 8000
```

Agarda a que vexas:

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Paso 2: Arrancar o Frontend (Flutter)

Abre un novo terminal e executa:

```
Alba@SanMamede-Apple-01 ScanGo % cd ScanGo/frontend  
flutter run -d chrome --web-port 3000
```

Agarda a que vexas:

```
webdev server listening on http://localhost:3000, serving web/
```

Paso 3: Abrir a aplicación

Vai a <http://localhost:3000> no navegador.

9. Verificación completa do sistema

Para comprobar que todo funciona correctamente, fai estas probas:

Proba 1: Rexistrarse como novo usuario

1. Na pantalla de login, busca a opción "¿Non tes conta?" ou similar
2. Fai clic en "Rexistro"
3. Introduce:
 - Nome: Test User
 - Email: test@test.com
 - Contraseña: test1234
4. Pulsa "Rexistrarse"
5. Deberías recibir un mensaje de confirmación

Proba 2: Iniciar sesión

1. Volve á pantalla de login
2. Introduce:
 - Email: alba@scango.com
 - Contraseña: password123
3. Pulsa "Iniciar sesión"
4. Deberías ver a pantalla principal (catálogo ou carrito)

Proba 3: Escanear un produto

1. Na pantalla principal, busca a opción "Escanear" ou "Engadir produto"
2. Introduce un código QR de proba: PROD001

3. O produto Leite enteiro 1L debe aparecer e engadirse ao carrito
4. Verifica que o prezo (€1,29) aparece correctamente

Proba 4: Ver o carrito

1. Fai clic en "Carrito" ou "Ver carrito"
2. Deberías ver o produto que engadiches con:
 - Nome: Leite enteiro 1L
 - Prezo: €1,29
 - Cantidade: 1
 - Total: €1,29

Proba 5: Verificar API desde Swagger (opcional)

Vai a <http://127.0.0.1:8000/docs> e proba:

1. GET /productos/ → Deberías ver os 8 produtos creados
2. GET /productos/escanear/PROD002 → Deberías ver "Pan integral 500g"
3. GET /usuarios/1 → Deberías ver os datos de Alba

Se todas as probas funcionan, parabéns! ScanGo está completamente operativo.

10. Parada do sistema

Para deter correctamente a aplicación:

Detener o Frontend (Flutter)

No terminal de Flutter, pulsa:

```
q
```

O terminal mostrará:

```
Exiting
```

Detener o Backend (FastAPI)

No terminal de FastAPI, pulsa:

```
CTRL+C
```

Ou en Mac:

```
Cmd+C
```

O terminal mostrará:

```
Shutdown complete
```

É importante facelo así xa que:

- Ctrl+C / q: Permite aos servizos pechar as conexións abertas de forma ordenada
- Matar o terminal: Pode deixar files/sockets abertos no sistema, causando conflitos máis tarde

Importante sempre detén o frontend primeiro, e o backend despois.

11. Resolución de problemas frecuentes

Se algo non funciona, consulta esta sección. A maioría de erros teñen solución sinxela.

→ Erro 1: "Address already in use 0.0.0.0:8000"

Síntoma: Ao executar o backend, aparece este erro.

Causa: O porto 8000 xa está en uso por outro proceso.

Solución:

Opción A: Usar un porto diferente

```
Alba@SanMamede-Apple-01 ScanGo % python3 -m uvicorn app.main:app --reload --port 8001
```

Despois, na aplicación Flutter, cambia a URL de localhost:8000 a localhost:8001.

Opción B: Liberar o porto 8000

- En Mac/Linux:

```
Alba@SanMamede-Apple-01 ScanGo % lsof -i :8000
kill -9 <PID>
```

- En Windows:

```
netstat -ano | findstr :8000
taskkill /PID <PID> /F
```

→ Erro 2: "Address already in use 127.0.0.1:3000"

Síntoma: Ao executar Flutter, aparece este erro.

Causa: O porto 3000 xa está en uso.

Solución:

```
Alba@SanMamede-Apple-01 ScanGo % flutter run -d chrome --web-port 3001
```

→ Erro 3: "Failed to connect to server"

Síntoma: A aplicación Flutter abre pero non podes iniciar sesión ou ver produtos.

Causa: A aplicación Flutter non consegue contactar co backend.

Solución:

1. Verifica que o backend está executándose en <http://127.0.0.1:8000>
2. Proba manualmente accedendo a <http://127.0.0.1:8000/docs> no navegador
3. Se non carga, o backend non está activo; volve ao paso de "Instalación do backend"
4. Verifica que a URL no código Flutter coincide (debe ser <http://localhost:8000>)

→ Erro 4: "flutter command not found"

Síntoma: Ao executar flutter run, aparece "command not found".

Causa: Flutter non está instalado ou non está no PATH do sistema.

Solución:

1. Descarga Flutter desde <https://flutter.dev/docs/get-started/install>
2. Descomprime nunha carpeta (ex: ~/flutter)
3. Engade ao PATH:

- En Mac/Linux (engade a ~/.zshrc ou ~/.bash_profile):

```
Alba@SanMamede-Apple-01 ScanGo % export PATH="$PATH:~/flutter/bin"
```

Despois:

```
Alba@SanMamede-Apple-01 ScanGo % source ~/.zshrc
flutter --version
```

- En Windows:
 1. Abre "Variables de entorno" (Environment Variables)
 2. Crea unha nova variable FLUTTER_HOME apuntando á carpeta de Flutter
 3. Engade %FLUTTER_HOME%\bin ao PATH
 4. Abre PowerShell e executa flutter --version

→ Erro 5: "pip: command not found"

Síntoma: Ao executar pip install, aparece que non existe o comando.

Solución: Usa pip3 en lugar de pip:

```
Alba@SanMamede-Apple-01 ScanGo % pip3 install -r requirements.txt
```

- Ou en Windows:

bash

python -m pip install -r requirements.txt

→ Erro 6: "No module named 'app'"

Síntoma: Ao executar python3 -m uvicorn app.main:app, aparece este erro.

Causa: Estás noutra carpeta ou o contorno virtual non está activado.

Solución:

1. Verifica que estás en ScanGo/backend:

```
Alba@SanMamede-Apple-01 ScanGo % pwd # (ou "cd" en Windows para ver a ruta actual)
```

2. Verifica que o contorno virtual está activado (debe haber (venv) ao inicio do terminal)

3. Se non está activado:

```
Alba@SanMamede-Apple-01 ScanGo % source venv/bin/activate # (Mac/Linux)
# ou .\venv\Scripts\Activate.ps1 (Windows)
```

→ Erro 7: "Database is locked"

Síntoma: Ao executar datosProba.py, aparece erro de base de datos.

Causa: O backend está activo usando a base de datos.

Solución:

1. Abre a base de datos en varios procesos á vez
2. Antes de executar datosProba.py, detén o backend (Ctrl+C)
3. Executa datosProba.py
4. Volve a arrancar o backend

→ Erro 8: "CORS error: Access denied"

Síntoma: A aplicación carga pero aparece erro de CORS no navegador (F12 → Console).

Causa: O backend non deixa peticións do frontend.

Solución: Verifica que en app/main.py está habilitado CORS:

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # calquerorixen
    allow_credentials=True,
    allow_methods=["*"], # Permite GET, POST, DELETE
    allow_headers=["*"],
)

```

Se non está, engádeo e reinicia o backend.

12.Despregamento en produción

ScanGo está preparado para executarse en entorno local de desenvolvemento (como se describe neste manual). Para produción, require configuracións adicionais.

Diferenzas principais

Aspecto	Desenvolvemento	Produción
Base de datos	SQLite (ficheiro local)	PostgreSQL (servidor)
Servidor web	Uvicorn local	Render.com (ou similar)
Protocolo	HTTP	HTTPS (seguro)
Dominio	localhost:8000	dominio.com
Reload automático	Sí (--reload)	Non

Pasos para produción (resumen)

1. **Crear conta en Render.com:** <https://render.com>
2. **Conectar o repositorio GitHub:** Render sincroniza automaticamente
3. **Configurar variables de entorno:** DATABASE_URL (PostgreSQL)
4. **Crear base de datos PostgreSQL en Render**
5. **Facer deploy:** Render compila e lanza o código automaticamente

Máis información

Para detalles completos sobre despregamento, consulta:

- [Render Deploy Guide](#)
- Documentación oficial de Flask/FastAPI para produción

Este manual está enfocado en desenvolvemento local. O despregamento en produción vai máis alá do alcance de instalación inicial.

Conclusión e seguintes pasos

Instalaches e verificaches ScanGo con éxito.

A partir de agora, cada vez que necesites usar a aplicación:

1. Abre dous terminales
2. Executa o backend (`python3 -m uvicorn app.main:app --reload --port 8000`)
3. Executa o frontend (`flutter run -d chrome --web-port 3000`)
4. Abre o navegador en `http://localhost:3000`

Se aparece algún erro non contemplado aquí, verifica:

- Os logs do terminal (conteñen pistas valiosas)
- Os logs do navegador (F12 → Console en Chrome)
- A documentación oficial das ferramentas (Python, Flutter, FastAPI)